

## 2.) Modeling and Simulation of IIR Filter using Direct Form Implementation:

### • Introduction:

- An Infinite Impulse Response (IIR) filter is a type of digital filter in which the output depends on both current and past input samples as well as previous output values.
- Unlike FIR filters, IIR filters use feedback, which makes them recursive in nature.
- The general form of an IIR filter is given by:

$$y(n) = \sum b_k x(n - k) + \sum a_k y(n - k)$$

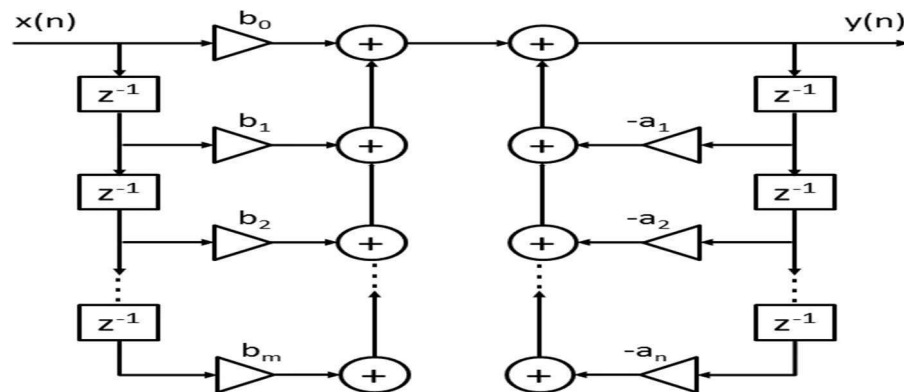
- In this project, the implemented IIR filter follows the equation:

$$y(n) = 2x(n) + 2x(n - 1) + 2x(n - 2) + 2x(n - 3) + y(n - 1)$$

- The presence of the feedback term  $y(n - 1)$  allows the filter to have an infinite impulse response.
- IIR filters are computationally efficient and require fewer resources compared to FIR filters for achieving similar performance.

### • Direct Form I Implementation of IIR Filter:

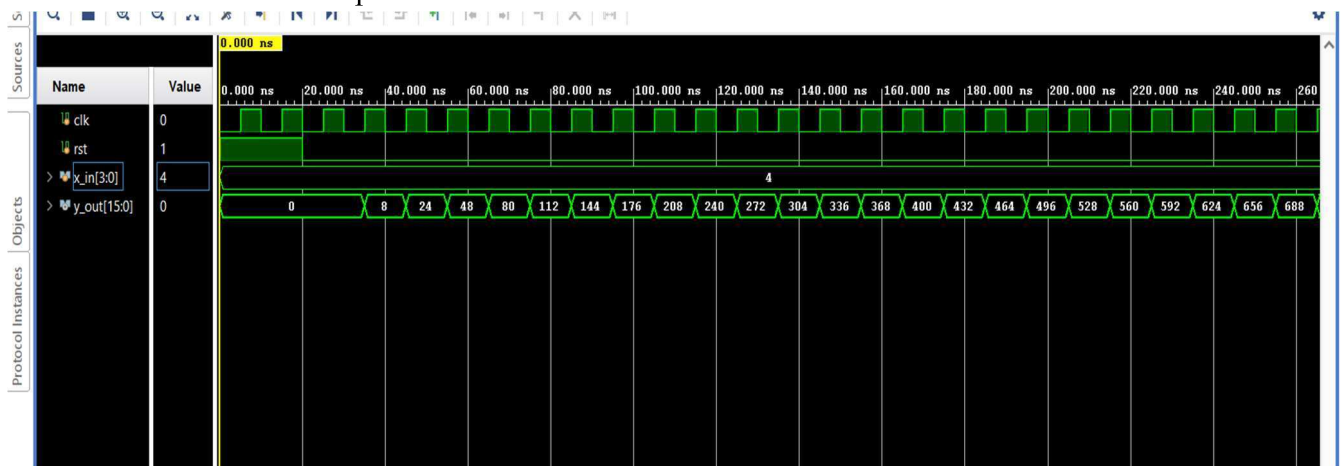
- Direct Form I is a commonly used structure for implementing IIR filters.
- It consists of two main sections: the feedforward (FIR) part and the feedback part.
- The feedforward section processes the current and past input samples, while the feedback section uses previous output values.
- Separate delay elements are used for input samples and output samples.
- The outputs of both sections are combined using adders to produce the final output.
- This structure directly follows the mathematical equation of the IIR filter.
- It is simple to understand and easy to implement with hardware and simulation.



**Fig 3.1) Direct Form-1 structure of IIR Filter**

- **Simulation of the Proposed IIR Filter:**

- The simulation of the proposed IIR filter is carried out to verify its functional correctness.
- A constant input is applied to observe the recursive behavior of the filter.
- The waveform shows that the output depends on both current inputs and previous outputs due to the feedback term.
- Unlike FIR filters, the output does not settle immediately and continues to evolve over successive clock cycles.
- The recursive nature of the filter results in an accumulating output response.
- The simulation results match the expected theoretical output, confirming correct implementation of the IIR filter.



**Fig 3.2) Simulation Results of IIR Filter**

- **Output Results of IIR Filter:**

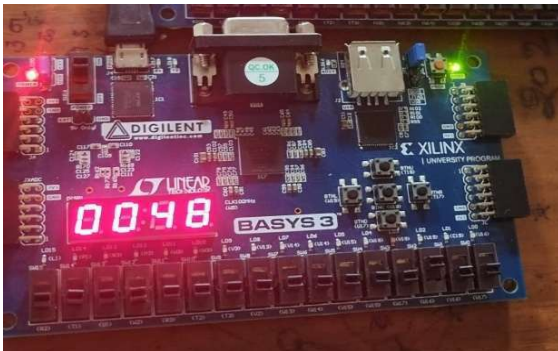
- The output of the IIR filter is obtained from the hardware implementation on the FPGA.
- A constant input value is applied, and the output is observed for each clock cycle.
- The output sequence shows a cumulative behavior due to the presence of the feedback term  $y(n - 1)$ .
- Unlike FIR filters, the output keeps increasing over successive clock cycles, demonstrating the recursive nature of the IIR filter.
- The observed output values match the expected theoretical sequence, confirming correct hardware implementation.
- It is observed that when the output exceeds the displayable range, the values wrap around due to finite bit-width representation.
- Since the display (LCD/7-segment) cannot represent large values beyond 256, the output appears in wrapped form (modulo representation).



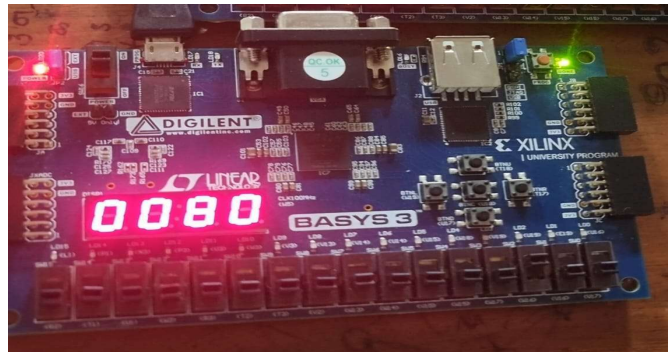
**Fig 3.3) First Output of an IIR filter**



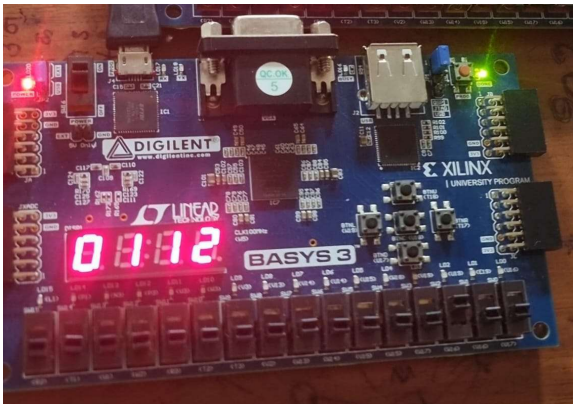
**Fig 3.4) Second Output of an IIR filter**



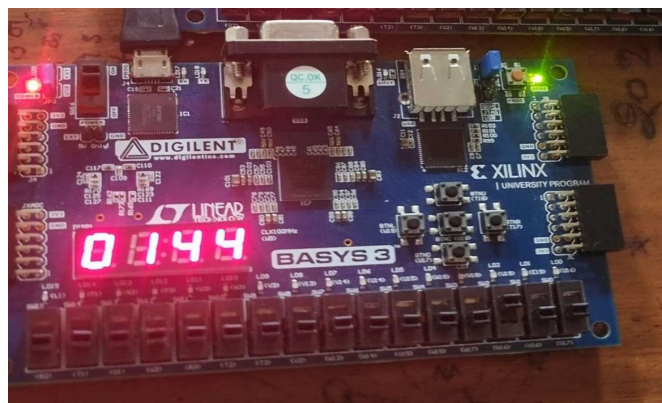
**Fig 3.5) Third Output of an IIR filter**



**Fig 3.6) Fourth Output of an IIR filter**



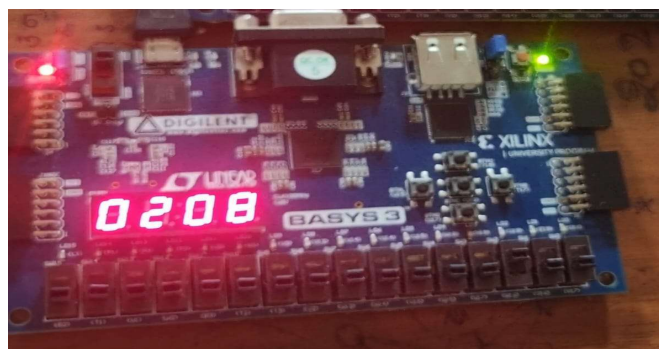
**Fig 3.7) Fifth Output of an IIR filter**



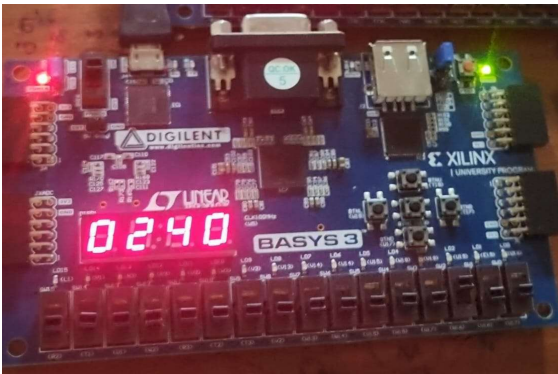
**Fig 3.8) Sixth Output of an IIR filter**



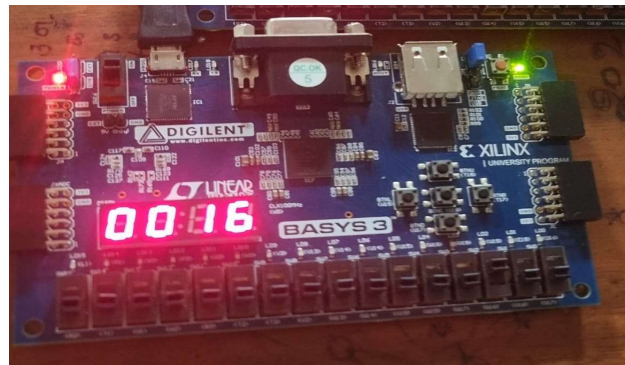
**Fig 3.9) Seventh Output of an IIR filter**



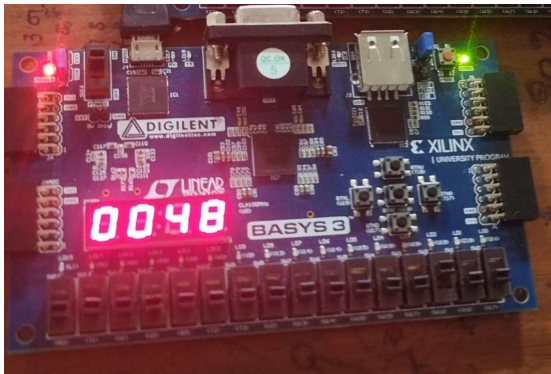
**Fig 3.10) Eighth Output of an IIR filter**



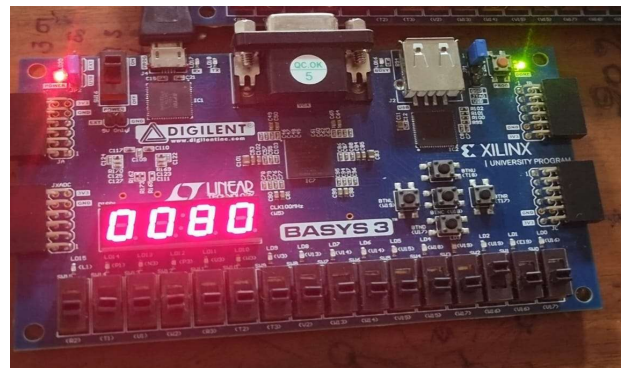
**Fig 3.11) Ninth Output of an IIR filter effect)**



**Fig 3.12) Tenth Output of an IIR filter (wrap around**



**Fig 3.13) Eleventh Output of an IIR filter effect)**



**Fig 3.14) Twelfth Output of an IIR filter (wrap around**

- **Timing Analysis of the IIR Filter:**

- Timing analysis is performed using the Vivado timing summary report.
- WNS, WHS, and WPWS are checked to verify timing performance.
- The feedback structure introduces a longer critical path in the design.
- No retiming is applied, so register placement is not optimized.
- This results in higher propagation delays in the combinational path.
- The design operates correctly if slack values remain positive.
- Overall timing performance is limited by the long critical path.

| Design Timing Summary                          |                                  |                                                   |
|------------------------------------------------|----------------------------------|---------------------------------------------------|
| Setup                                          | Hold                             | Pulse Width                                       |
| Worst Negative Slack (WNS): 6.336 ns           | Worst Hold Slack (WHS): 0.259 ns | Worst Pulse Width Slack (WPWS): 4.500 ns          |
| Total Negative Slack (TNS): 0.000 ns           | Total Hold Slack (THS): 0.000 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 0                 | Number of Failing Endpoints: 0   | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 89                  | Total Number of Endpoints: 89    | Total Number of Endpoints: 47                     |
| All user specified timing constraints are met. |                                  |                                                   |

**Fig 3.15) Design Timing Summary of IIR filter**

| Clock Summary |               |             |                 |
|---------------|---------------|-------------|-----------------|
| Name          | Waveform      | Period (ns) | Frequency (MHz) |
| CLK100MHZ     | {0.000 5.000} | 10.000      | 100.000         |

**Fig 3.16) Clock Period of IIR filter**

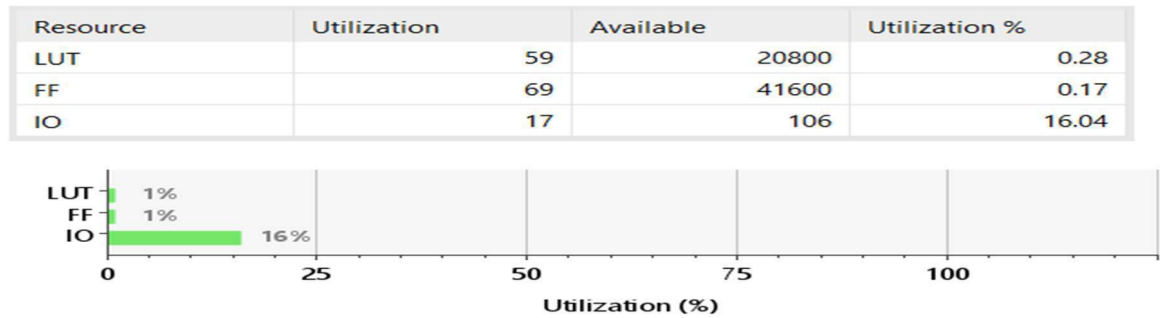
Critical Path delay = Clock Period-WNS=10ns-6.336ns=3.664 ns

Frequency =  $1/T_{cp}=1/3.664\text{ns}=272.92\text{ MHz}$

- **Resource Utilization of the IIR Filter:**

- The resource utilization of the implemented IIR filter is analyzed using the Vivado synthesis report.
- The report provides details of FPGA resources such as lookup tables (LUTs), slice registers, and other logic elements used in the design.
- LUTs are used for implementing combinational logic, while slice registers are used for storing intermediate and feedback values.
- The recursive structure of the IIR filter leads to higher utilization compared to purely feedforward architecture.
- Multipliers and adders contribute significantly to the overall hardware usage.
- The results reflect an efficient FPGA mapping of the IIR filter design.
- Overall, the design achieves correct functionality with acceptable resource consumption.

## Summary



**Fig 3.17) Overall Resource Utilization Summary of the Design**

| Name       | Slice LUTs<br>(20800) | Slice Registers<br>(41600) | Slice<br>(8150) | LUT as Logic<br>(20800) | Bonded IOB<br>(106) | BUFGCTRL<br>(32) |
|------------|-----------------------|----------------------------|-----------------|-------------------------|---------------------|------------------|
| top_basys3 | 59                    | 69                         | 34              | 59                      | 17                  | 1                |

**Fig 3.18) Hierarchical Resource Utilization of the Top Module**

### • **Power Analysis of the IIR Filter:**

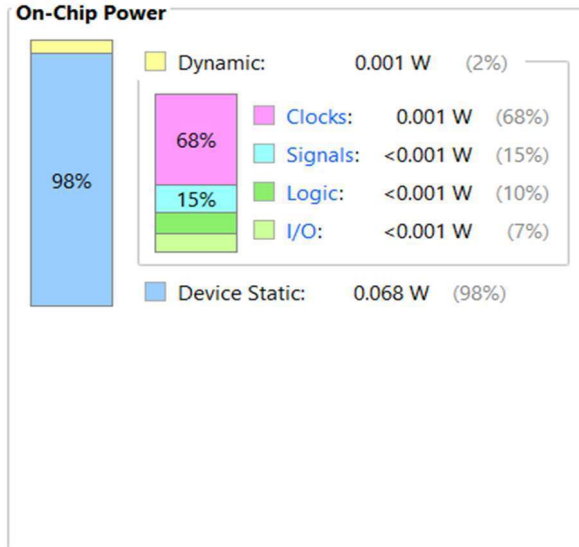
- The power analysis of the implemented IIR filter is carried out using the Vivado power report tool.
- The report estimates both static and dynamic power consumption of the design.
- Static power is mainly due to leakage currents in the FPGA device.
- Dynamic power arises from switching activity in logical elements, registers, and clock networks.
- The feedback nature of the IIR filter contributes to continuous signal activity within the circuit.
- The results show moderate power consumption based on the implemented architecture and operating frequency.
- Overall, the design is power-efficient and suitable for FPGA-based digital signal processing applications.

## Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

**Total On-Chip Power:** **0.07 W**  
**Design Power Budget:** **Not Specified**  
**Process:** **typical**  
**Power Budget Margin:** **N/A**  
**Junction Temperature:** **25.3°C**  
Thermal Margin: 74.7°C (14.8 W)  
Ambient Temperature: 25.0 °C  
Effective  $\theta_{JA}$ : 5.0°C/W  
Power supplied to off-chip devices: 0 W  
Confidence level: **Low**

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity



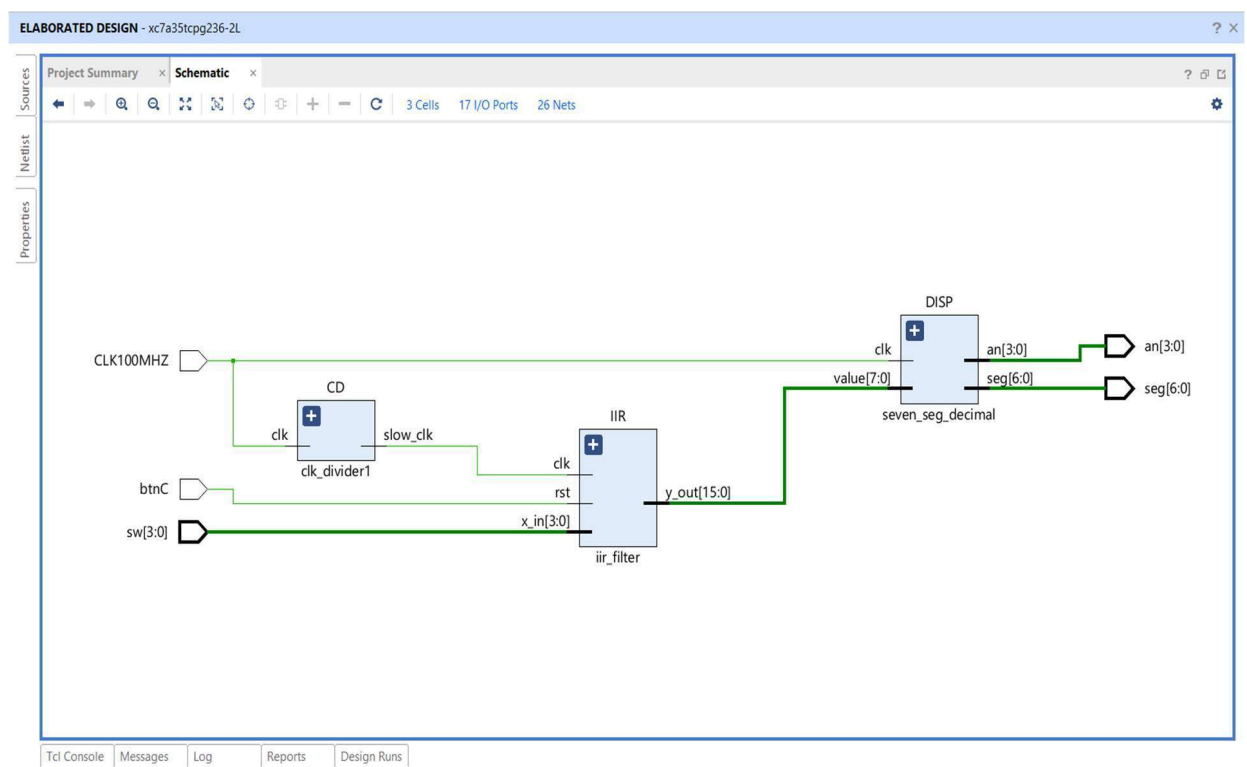
**Fig 3.19) On-Chip Power Distribution of the IIR Filter**

| Settings                      | Supply Source | Voltage (V) | Total (A) | Dynamic (A) | Static (A) | Budget (A)  | Margin (A) |
|-------------------------------|---------------|-------------|-----------|-------------|------------|-------------|------------|
| Summary (0.07 W, Margin: N/A) | Vccint        | 1.000       | 0.011     | 0.001       | 0.010      | Unspecified | NA         |
| Power Supply                  | Vccaux        | 1.800       | 0.013     | 0.000       | 0.013      | Unspecified | NA         |
| Utilization Details           | Vcco33        | 3.300       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Hierarchical (0.001 W)        | Vcco25        | 2.500       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Clocks (0.001 W)              | Vcco18        | 1.800       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Signals (<0.001 W)            | Vcco15        | 1.500       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Data (<0.001 W)               | Vcco135       | 1.350       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Set/Reset (<0.001 W)          | Vcco12        | 1.200       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Logic (<0.001 W)              | Vccaux_io     | 1.800       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| I/O (<0.001 W)                | Vccbram       | 1.000       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
|                               | MGTA Vcc      | 1.000       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
|                               | MGTA Vtt      | 1.200       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
|                               | Vccadc        | 1.800       | 0.020     | 0.000       | 0.020      | Unspecified | NA         |

**Fig 3.20) Power Supply Breakdown of the IIR Filter Design**

- **Elaborated Design of the Hardware Circuit of IIR Filter:**

- The hardware design of the IIR filter is based on a recursive structure that uses both present input samples and previous output samples.
- The design includes delay elements (registers) to store past input and output values required for feedback operation.
- Combinational logic is used to perform multiplication and addition operations for both feedforward and feedback paths.
- The output is generated using a combination of current input and weighted past outputs, forming a feedback loop.
- The overall architecture is implemented in FPGA using adders, multipliers, and registers in a structured dataflow manner.
- The block diagram of the implemented IIR filter is shown below.



**Fig 3.21) Elaborated Design of the IIR Filter Design**

- **Conclusion:**

- The IIR filter has been successfully designed and implemented on the FPGA using recursive (feedback-based) architecture.
- The results obtained from simulation and hardware analysis confirm the correct functionality of the design.
- The design meets the required performance in terms of timing, power, and resource utilization as per the synthesis reports.
- This implementation provides a foundation for further optimizations such as retiming or pipelining to enhance performance

### 3.) **Modeling and Simulation of Retimed IIR Filter using Direct Form Implementation:**

- **Introduction:**

- The 4-tap IIR filter is designed to improve speed and timing performance of a basic recursive filter.
- The filter consists of both feedback components.
- The difference equation used is:

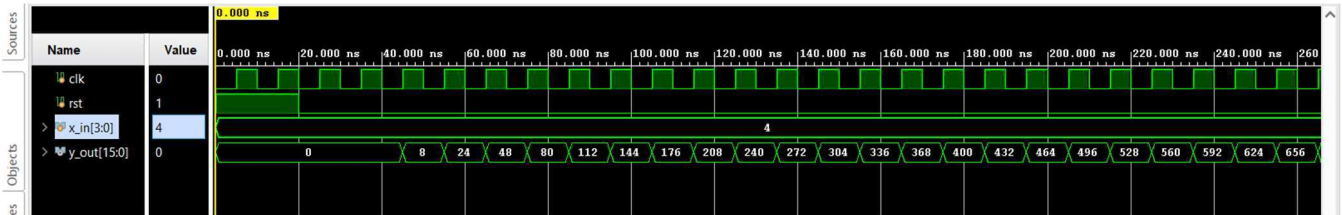
$$y(n)=2x(n)+2x(n-1) +2x(n-2) +2x(n-3) +y(n-1)$$

- The feedforward coefficients are 2, and the feedback coefficient is 1.
- The output depends on both current/past input samples and the previous output sample.
- Retiming is applied by moving registers into the critical paths of the design.
- This helps in breaking long combinational delay paths in both forward and feedback sections.
- The computation is distributed across multiple clock cycles using register balancing.
- This reduces the critical path delay and improves the maximum operating frequency.
- The design introduces an initial latency of a few clock cycles.
- After the initial delay, the filter produces one output per clock cycle.
- This retimed structure improves overall hardware efficiency compared to basic IIR implementation.

- **Simulation of the proposed Retimed IIR filter:**

- The simulation of the proposed 4-tap IIR filter is performed to verify its functional correctness and timing behavior.
- The waveform shows that the output is available after an initial delay of one clock cycle due to register balancing in the design.
- This single-cycle delay represents the internal computation and storage of intermediate values in the retimed structure.

- After this initial delay, the filter produces one valid output per clock cycle.
- The output sequence matches the expected theoretical IIR response, confirming correct implementation of both feedforward and feedback paths.
- The retimed structure ensures proper timing closure by reducing long combinational paths in the recursive loop.
- This improves overall operating frequency while maintaining the correct filter functionality.



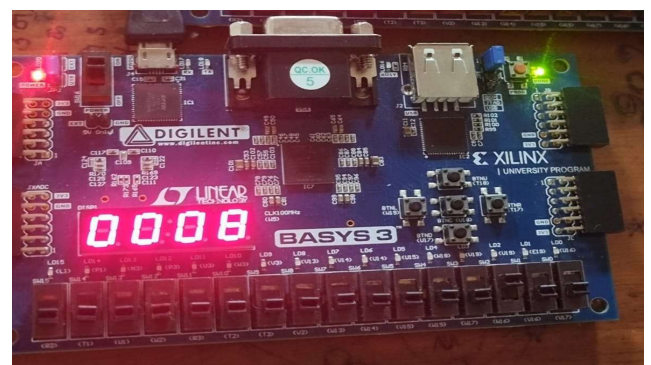
**Fig 4.1) Simulation of the Retimed IIR Filter Design**

• **Output Results of Retimed IIR Filter:**

- The IIR filter output is obtained from FPGA implementation with a constant input applied.
- The output appears after an initial delay of the one-clock cycle due to register balancing.
- After this, one valid output is produced every clock cycle.
- The output shows cumulative behavior due to the feedback term  $y(n - 1)$ .
- The response matches the expected theoretical IIR behavior, confirming correct implementation.
- Output wrapping is observed when values exceed display range due to finite bit-width representation.
- The display shows modulo behavior as it cannot represent large values beyond its limit.



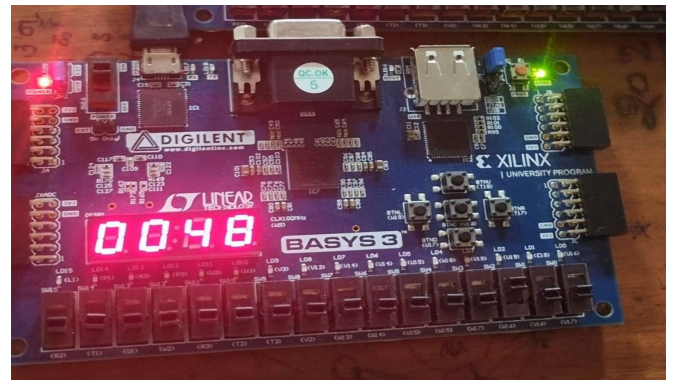
**Fig 4.2) Output During First Clock Cycle**



**Fig 4.3) First Valid Output at the 3rd Clock**



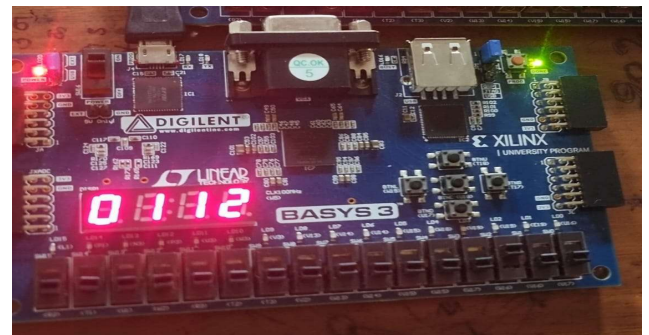
**Fig 4.4) Second Valid Output at the 4th Clock Cycle**



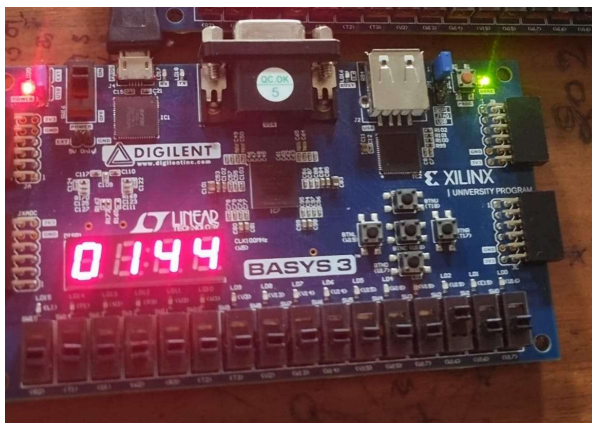
**Fig 4.5) Third Valid Output of the at the 5th Clock**



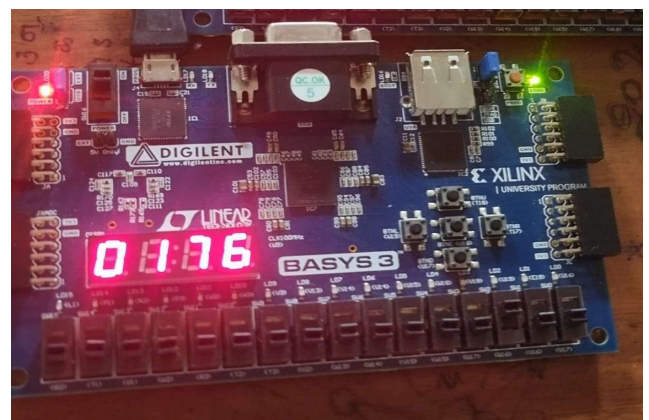
**Fig 4.6) Fourth Valid Output at the 6th Clock Cycle**



**Fig 4.7) Fifth Valid Output at the 7th Clock**



**Fig 4.8) Sixth Valid Output at 8th Clock Cycle**



**Fig 4.9) Seventh Valid Output at the 9th Clock**

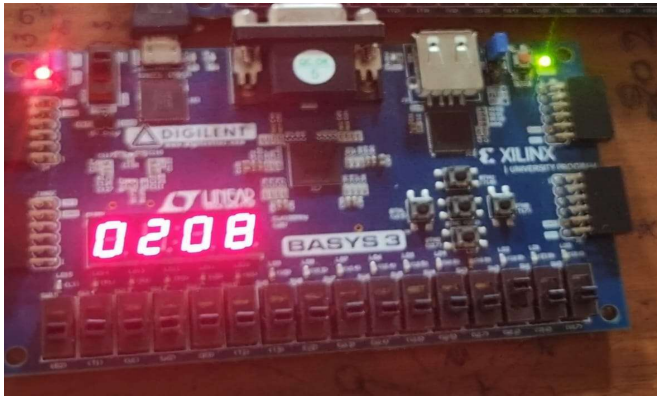


Fig 4.10) Eighth Valid at the 10th Clock Cycle

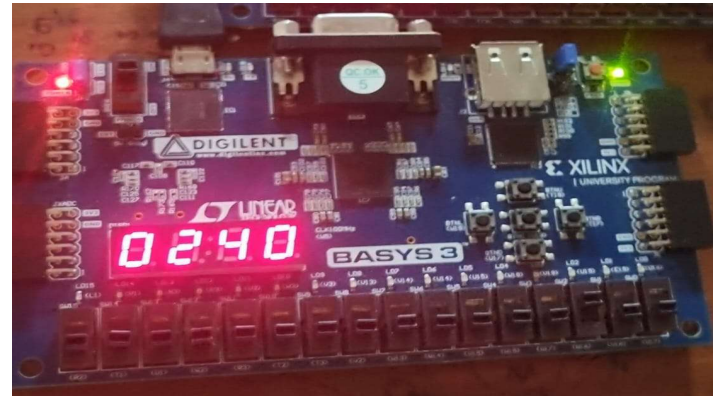


Fig 4.11) Ninth Valid Output at the 11th Clock

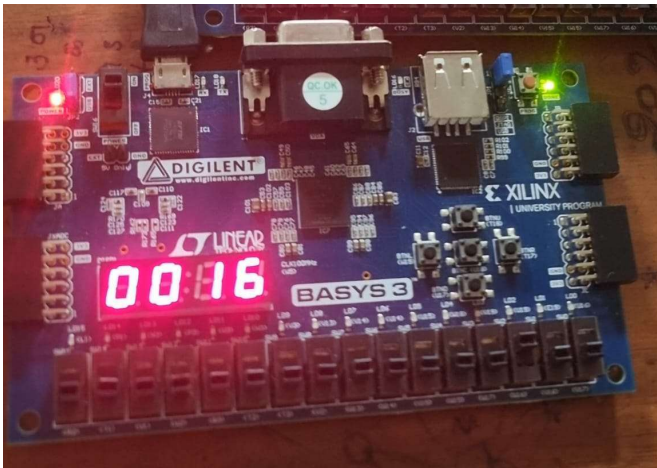


Fig 4.12) Tenth Valid Output at the 12th Clock Cycle  
(wrap around effect)

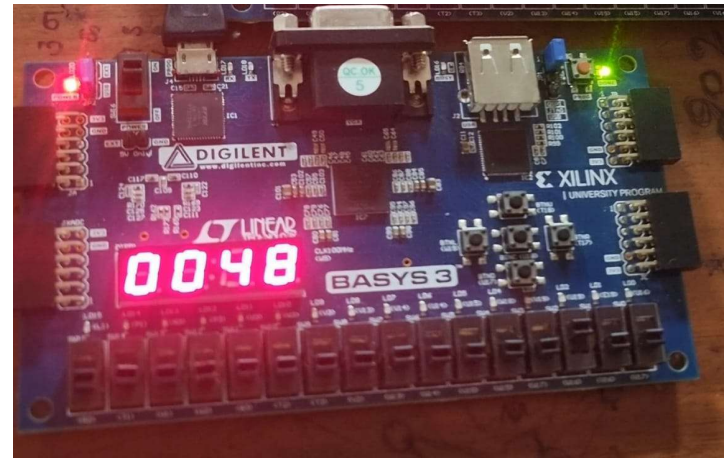


Fig 4.13) Eleventh Valid Output at the 13th Clock  
(wrap around effect)

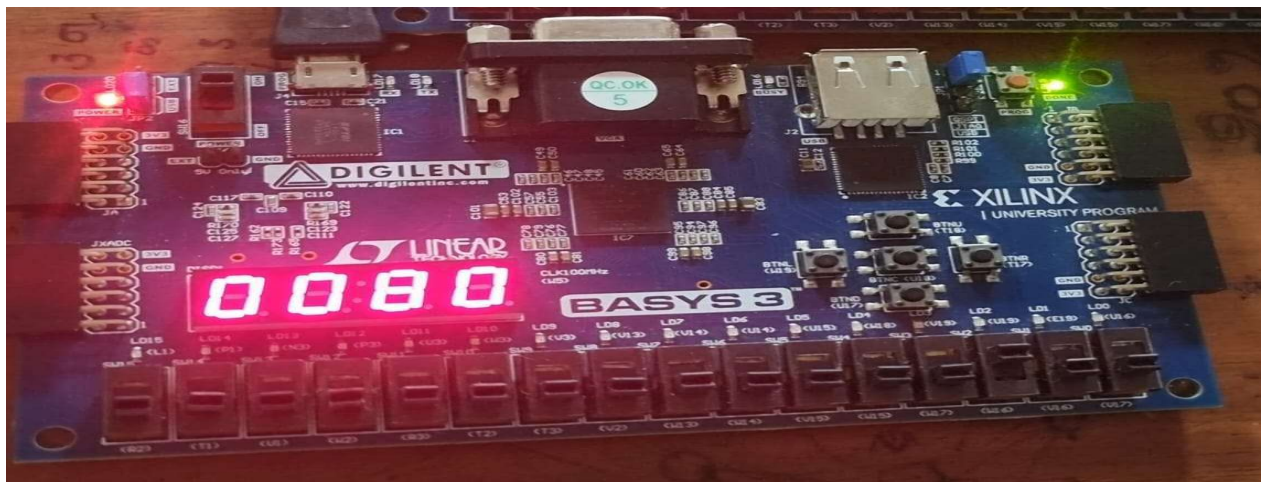


Fig 4.14) Twelfth Valid Output of the Retimed IIR Filter at the 14th Clock Cycle  
(wrap around effect)

- **Timing Analysis of the Retimed IIR Filter:**

- Timing analysis is performed using the Vivado timing summary report.
- WNS, WHS, and WPWS values are checked to evaluate timing performance.
- Retiming redistributes registers to balance the feedback path.  
This reduces the effective critical path delay compared to the non-retimed design.
- The design achieves improved timing slack due to optimized register placement.
- No functional change is observed in the filter response after retiming.  
Overall timing performance is improved, making the design more efficient for high-speed operation.

| Design Timing Summary                          |                                  |                                                   |
|------------------------------------------------|----------------------------------|---------------------------------------------------|
| Setup                                          | Hold                             | Pulse Width                                       |
| Worst Negative Slack (WNS): 6.451 ns           | Worst Hold Slack (WHS): 0.095 ns | Worst Pulse Width Slack (WPWS): 4.500 ns          |
| Total Negative Slack (TNS): 0.000 ns           | Total Hold Slack (THS): 0.000 ns | Total Pulse Width Negative Slack (TPWS): 0.000 ns |
| Number of Failing Endpoints: 0                 | Number of Failing Endpoints: 0   | Number of Failing Endpoints: 0                    |
| Total Number of Endpoints: 89                  | Total Number of Endpoints: 89    | Total Number of Endpoints: 47                     |
| All user specified timing constraints are met. |                                  |                                                   |

**Fig 4.15) Design Timing Summary of Retimed IIR filter**

| Name      | Waveform      | Period (ns) | Frequency (MHz) |
|-----------|---------------|-------------|-----------------|
| CLK100MHZ | {0.000 5.000} | 10.000      | 100.000         |

**Fig 4.16) Clock Period of Retimed IIR filter**

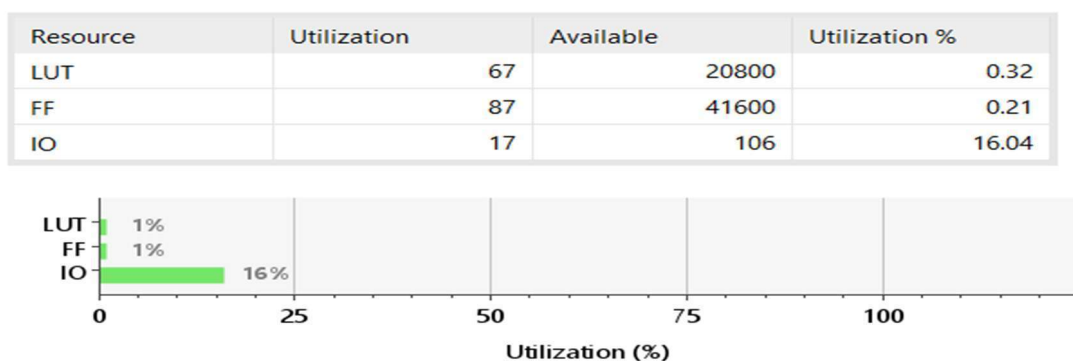
Critical Path delay = Clock Period-WNS=10ns-6.451ns=3.549 ns

Frequency = 1/Tcp=1/3.549ns=281.76 MHz

- **Resource Utilization of the Retimed IIR Filter:**

- The resource utilization of the retimed IIR filter is analyzed using the Vivado synthesis report.
- The report provides details of FPGA resources such as LUTs, slice registers, and other logic elements used in the design.
- LUTs are used for implementing combinational logic, while slice registers store intermediate and feedback values.
- Retiming redistributes registers across the design, leading to a more balanced hardware structure.
- This may slightly increase slice registers due to added pipeline balancing registers.
- The combinational logic remains unchanged, so LUT usage is nearly the same as the non-retimed design.
- Overall, the design achieves improved timing performance with minimal impact on total resource utilization.

#### Summary



**Fig 4.17) Overall Resource Utilization Summary of the Design**

| Hierarchy                |                       |                            |                     |                 |                         |                     |                  |
|--------------------------|-----------------------|----------------------------|---------------------|-----------------|-------------------------|---------------------|------------------|
| Name                     | Slice LUTs<br>(20800) | Slice Registers<br>(41600) | F7 Muxes<br>(16300) | Slice<br>(8150) | LUT as Logic<br>(20800) | Bonded IOB<br>(106) | BUFGCTRL<br>(32) |
| top_basys3               | 67                    | 87                         | 6                   | 44              | 67                      | 17                  | 2                |
| IIR (iir_filter_retimed) | 52                    | 41                         | 6                   | 24              | 52                      | 0                   | 0                |

**Fig 4.18) Hierarchical Resource Utilization of the Top Module**

- **Power Analysis of the Retimed IIR Filter:**

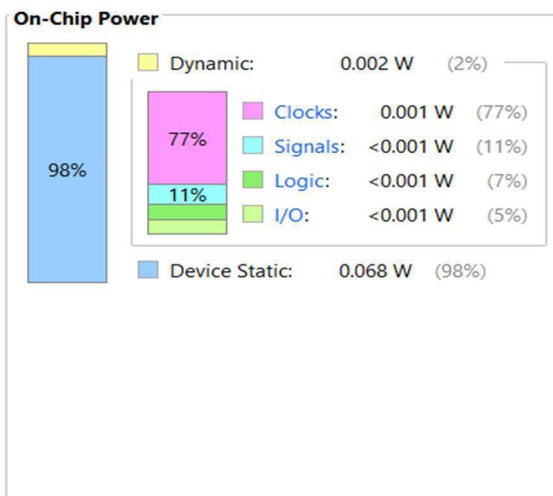
- The power analysis of the retimed IIR filter is carried out using the Vivado power report tool.
- The report estimates both static and dynamic power consumption of the design.
- Static power is mainly due to leakage currents in the FPGA device.
- Dynamic power arises from switching activity in logic elements, registers, and clock distribution networks.
- Retiming increases register usage slightly, which can cause a small rise in clock network power.
- However, improved balancing reduces long combinational switching activity in the critical path.
- Overall, the design maintains power efficiency with a slight trade-off between timing improvement and dynamic power.

#### Summary

Power analysis from Implemented netlist. Activity derived from constraints files, simulation files or vectorless analysis.

|                                     |                      |
|-------------------------------------|----------------------|
| <b>Total On-Chip Power:</b>         | <b>0.07 W</b>        |
| <b>Design Power Budget:</b>         | <b>Not Specified</b> |
| <b>Process:</b>                     | typical              |
| <b>Power Budget Margin:</b>         | N/A                  |
| <b>Junction Temperature:</b>        | 25.3°C               |
| Thermal Margin:                     | 74.7°C (14.8 W)      |
| Ambient Temperature:                | 25.0 °C              |
| Effective $\theta_{JA}$ :           | 5.0°C/W              |
| Power supplied to off-chip devices: | 0 W                  |
| Confidence level:                   | Low                  |

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity



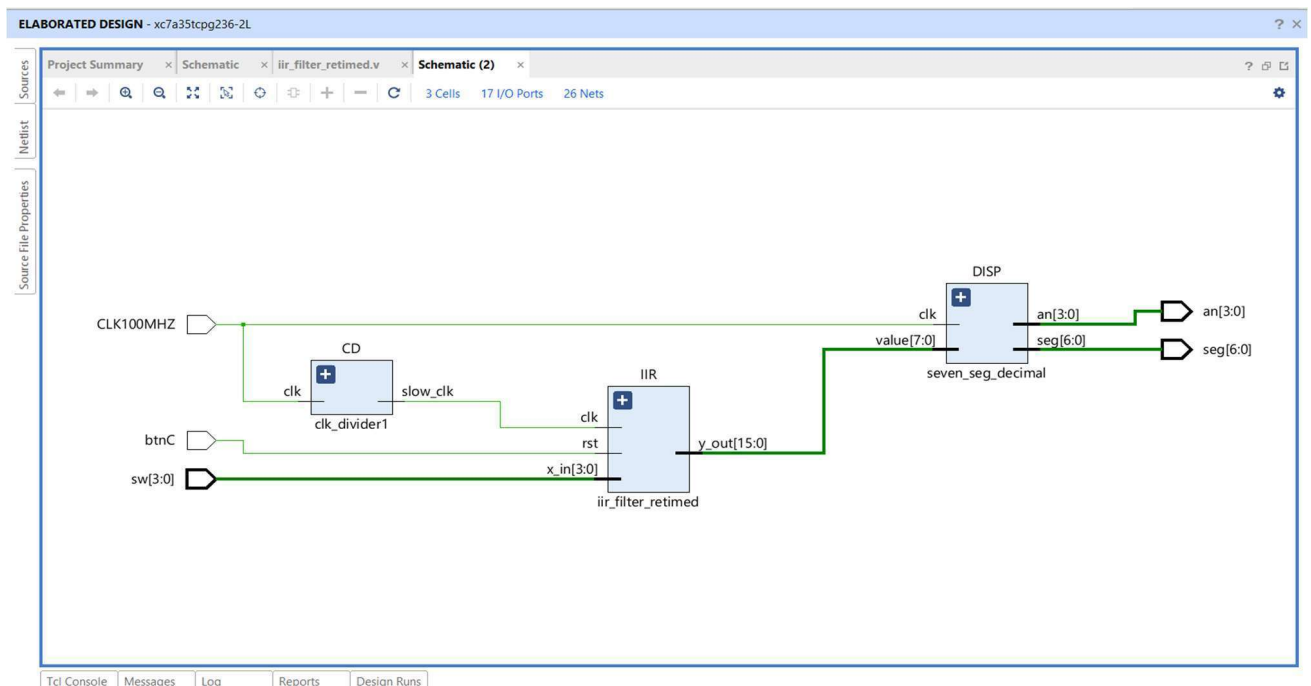
**Fig 4.19) On-Chip Power Distribution of the Retimed IIR Filter**

|                               |               |             |           |             |            |             |            |
|-------------------------------|---------------|-------------|-----------|-------------|------------|-------------|------------|
| Settings                      | Supply Source | Voltage (V) | Total (A) | Dynamic (A) | Static (A) | Budget (A)  | Margin (A) |
| Summary (0.07 W, Margin: N/A) | Vccint        | 1.000       | 0.011     | 0.002       | 0.010      | Unspecified | NA         |
| Power Supply                  | Vccaux        | 1.800       | 0.013     | 0.000       | 0.013      | Unspecified | NA         |
| Utilization Details           | Vcco33        | 3.300       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Hierarchical (0.002 W)        | Vcco25        | 2.500       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Clocks (0.001 W)              | Vcco18        | 1.800       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Signals (<0.001 W)            | Vcco15        | 1.500       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Data (<0.001 W)               | Vcco135       | 1.350       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Set/Reset (<0.001 W)          | Vcco12        | 1.200       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| Logic (<0.001 W)              | Vccaux_io     | 1.800       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
| I/O (<0.001 W)                | Vccbram       | 1.000       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
|                               | MGTAVcc       | 1.000       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
|                               | MGTAVtt       | 1.200       | 0.000     | 0.000       | 0.000      | Unspecified | NA         |
|                               | Vccadc        | 1.800       | 0.020     | 0.000       | 0.020      | Unspecified | NA         |

**Fig 4.20) Power Supply Breakdown of the Retimed IIR Filter Design**

- **Elaborated Design of the Hardware Circuit of Retimed IIR Filter:**

- The retimed IIR filter is based on a recursive structure that uses both current input samples and previous output samples.
- The design includes register-based delay elements to store past input and output values required for feedback operation.
- The feedforward computation is divided into multiple registered stages to reduce the combinational delay.
- Retiming is applied by relocating registers across the arithmetic path to balance the critical path.
- The output is generated by combining the computed feedforward result with the previous output in a feedback loop.
- The architecture is implemented on FPGA using adders, shifters, and registers in a structured pipelined dataflow manner.
- The block diagram of the implemented retimed IIR filter is shown below.



**Fig 4.21) Elaborated Design of the Retimed IIR Filter Design**

- **Comparison Between Non-Retimed and Retimed IIR Filter:**

| Parameter                  | IIR Filter                      | Retimed IIR Filter              |
|----------------------------|---------------------------------|---------------------------------|
| <b>Critical Path Delay</b> | <b>3.664 ns</b> <i>(higher)</i> | <b>3.549 ns</b> <i>(lower)</i>  |
| <b>Max Frequency</b>       | <b>273 MHz</b> <i>(lower)</i>   | <b>282 MHz</b> <i>(higher)</i>  |
| <b>Latency</b>             | <b>1 cycle</b> <i>(lower)</i>   | <b>2 cycles</b> <i>(higher)</i> |
| <b>Throughput</b>          | <b>1 output/cycle</b>           | <b>1 output/cycle</b>           |
| <b>LUT Utilization</b>     | <b>59</b>                       | <b>67</b>                       |
| <b>SLICE registers</b>     | <b>69</b> <i>(lower)</i>        | <b>87</b> <i>(higher)</i>       |
| <b>Dynamic Power</b>       | <b>1 mW</b> <i>(lower)</i>      | <b>2 mW</b> <i>(higher)</i>     |